# MSA

October University for Modern Sciences and Arts
Faculty of Engineering
Department of Mechatronics System Engineering

# SUMO ROBOT

**Prepared By**



| | |
|---|---|
| Jason Sameh Zarref | ID: 251086 |
| Youssef Mohamed Mostafa | ID: 255000 |
| Felobateer Akram Fawzi | ID: 255231 |
| Ibrahim Wael Abdelmonem | ID: 251112 |
| Mohamed Ahmed Imam | ID: 253440 |
| Mohamed Samer | ID: 253335 |
| Haneen Ahmed Nabih | ID: 251417 |

**Supervisor**

Dr. Amgad M. Bayoumy

# Table of Contents

# Chapter 1: Introduction

## 1.1 Background

In recent years, robotics has become an increasingly accessible field due to advancements in microcontrollers, wireless communication, and compact power solutions. Educational and hobbyist robotics projects have gained popularity in academic environments and maker communities worldwide.

The SUMO Robot is a mobile robotic system designed to demonstrate fundamental principles of embedded systems, wireless control, and mechatronics integration. The core of the robot is an ESP-series microcontroller that processes input, executes control logic, and communicates with external devices. Acting as the intermediary between the microcontroller and physical motion is a motor driver, which supplies controlled current to two DC motors mounted on a two-wheel drive platform.

The power supply consists of four 3.7 V batteries connected to provide reliable voltage and current necessary for motor and controller operation. This combination of hardware allows the robot to move, respond to commands, and serve as a practical tool for learning and experimentation in robotics design.

## 1.2 Problem Definition

Despite the growing interest in robotics education, many existing mobile robotic kits are either overly expensive or lack flexibility for customization and real-world experimentation. Learners and hobbyists often encounter challenges when integrating wireless communication, motor control, and power management on a single platform. Specifically, problems include:

- Limited control precision: standard robot kits may not allow programmable control using open-source environments.
- Power inefficiency: some designs fail to provide stable power, leading to reduced motor performance or controller resets.
- Lack of integration: communication modules and motor control components are not always designed to work together seamlessly.

The SUMO Robot project addresses these issues by combining an ESP microcontroller, a dedicated motor driver, and a suitable battery system to create an affordable, adaptable, and efficient mobile robotic platform.

## 1.3 Functional Requirements

The SUMO Robot is designed to perform the following functions:

1. **Wireless Command Reception —** the robot must receive movement commands (e.g., forward, backward, turn) via a wireless interface supported by the ESP microcontroller.
2. **Motor Control —** the system must drive two DC motors using the motor driver, translating control signals into motion with appropriate direction and speed.
3. **Stable Power Delivery —** the battery pack must supply sufficient voltage and current to both the ESP controller and the motors without significant voltage drop during operation.

4. **Directional Movement —** the robot must be able to move forward, backward, turn left, and turn right using differential wheel control.
5. **Robust Mechanical Design —** the two-wheel drive base must support the robot's weight and maintain balance during motion.

## 1.4 Project Specification

| Component | Specification |
|---|---|
| Controller | ESP-series microcontroller (e.g., ESP32 / ESP8266) |
| Motor Driver | Dual-channel motor driver (compatible with 2 DC motors) |
| Motors | 2 × DC motors (rated for low-voltage operation) |
| Wheels | 2 × compatible wheels |
| Power Supply | 4 × 3.7 V rechargeable batteries (lithium-ion or LiPo) |
| Communication | Wi-Fi / Bluetooth (based on the ESP module used) |
| Chassis | Lightweight platform supporting all components |
| Control Interface | Mobile app, web interface, or remote controller |
| Operating Voltage | Approx. 14.8 V (with appropriate regulation for ESP and motor driver) |

***Note:*** *if needed, a voltage regulator or DC-DC converter can be used to match the ESP's operating voltage requirements.*

## 1.5 Current Technology

Robotics platforms today range from simple educational kits to advanced autonomous systems. Common solutions in the current field include:

- **Arduino-based robots:** widely used in education, these robots use Arduino microcontrollers with basic motor shields for motion control. They are simple but can be limited in wireless capabilities without additional modules.
- **Raspberry Pi robots:** with a single-board computer at their core, these robots can perform complex processing and vision tasks but are often bulkier and require more power.
- **ESP-controlled robots:** the rise of ESP microcontrollers (such as ESP32 and ESP8266) has enabled low-cost robotic platforms with integrated Wi-Fi and Bluetooth, making remote control and IoT integration easier. They strike a balance between processing capability, power consumption, and connectivity.
- **Commercial kits:** products such as LEGO Mindstorms and VEX offer modular designs but at a higher cost and with limited openness for custom hardware/software extension.

Compared with these technologies, the SUMO Robot leverages the strengths of the ESP ecosystem, efficient power configuration, and a simple drive system to offer a flexible and affordable mobile platform suitable for learning, prototyping, and experimentation.

# Chapter 2: System Design

## 2.1 Discussion of Design

A DC motor was selected as the actuator for this project because it offers high starting torque, precise speed control, a compact size, and good performance across a range of load conditions.

An ESP32 was chosen as the microcontroller due to its low cost, high-performance dual-core processing, and built-in Bluetooth and Wi-Fi modules for wireless connectivity.

The system architecture connects the voltage source to the motor driver, which in turn supplies the two motors. The ESP32 microcontroller is connected wirelessly to a mobile phone, which sends the movement commands.

A 3D-printed chassis was used for the robot body. Although 3D printing costs more than a standard robot chassis, the resulting structure is significantly stronger. Similarly, rechargeable batteries were chosen over standard batteries; although more expensive upfront, they can be reused many times, making them more cost-effective over the project's lifetime.

## 2.2 Design Specification

### Electrical

Operating voltage: 14.8 V. Battery life for the four batteries: approximately 20–30 minutes of continuous operation.

### Mechanical

Footprint: 15 cm × 20 cm. Total weight: up to 1.5 kg.

### Software

Bluetooth response time: between 40 ms and 150 ms. Programming language: C++.

## 2.3 System Outline

The robot's control architecture follows the signal path shown below: the mobile phone and the motor driver both feed information to the ESP32 controller, which processes this input and issues commands to the DC motors (the actuators). The motors then produce the physical output — the robot's motion.

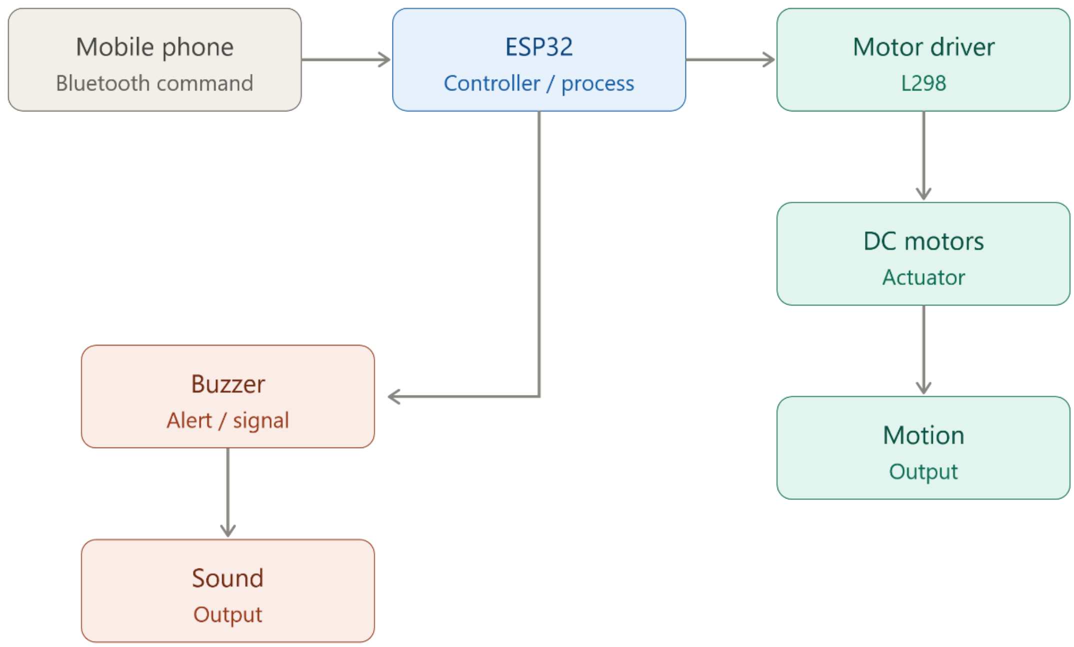



**Figure 2.1 — System-level signal and control flow**

# Chapter 3: Mechanical Design

The mechanical design of the robot is built around a differential-drive system powered by two high-torque DC motors. The chassis design prioritizes a low center of gravity and high traction to meet the requirements of the Sumo class.

## 3.1 Mechanical Design Calculations

The selection of the drive system was based on the need for high pushing torque and the ability to resist being pushed by opponents. The robot uses two miniature worm-gear DC motors rated at 12 V, with a gear ratio of 1:40 and a wheel diameter of 65 mm.

### 3.1.1 Torque and Pushing Force Analysis

The theoretical pushing force was calculated to confirm that the robot can overcome the friction of the dohyo (ring) and push opponents.

**Motor specifications:**

- Initial torque ($T_{Initial}$) = 6.4 kg·cm
- Wheel radius (r) = 32.5 mm = 0.0325 m

First, the torque is converted to SI units (newton-metres):

$$T\_Initial = 6.4\ kg \cdot cm \times 0.098 = 0.628\ N \cdot m$$

$$T\_Total = 2 \times 0.628 = 1.256\ N \cdot m$$

The theoretical linear pushing force is then:

$$F\_Push = T\_Total \div r = 1.256\ N \cdot m \div 0.0325\ m \approx 38.6\ N$$

So, the robot can push an opposing sumo robot of approximately 3 kg.

## 3.2 Mathematical Modeling

To implement precise control in the software, a mathematical model of the differential-drive system was established. This model relates the inputs (left and right wheel velocities) to the robot's pose in the global frame.

### 3.2.1 Differential Drive Kinematics

The robot's motion is defined by its position (x, y) and orientation (θ). The kinematic equations for the differential-drive system are derived below.

Let $V_R$ and $V_L$ be the linear velocities of the right and left wheels, and let L be the track width (the distance between the wheel centres).

**Linear velocity of the robot (V):**

$$V = (v_R + v_L) / 2$$

**Angular velocity of the robot (ω):**

The rate of rotation is determined by the difference in wheel speeds divided by the track width (L):

$$\omega = (v_R - v_L) / L$$

# Chapter 4: Electrical Design

This chapter presents the electrical design of the proposed system and explains how electrical power is generated, distributed, and utilized to ensure proper system operation. It focuses on the selection of electrical components, the configuration of the power supply, and the calculations required to guarantee reliable and safe performance.

The electrical system plays a critical role in operating the control unit and driving the DC motors through a suitable motor driver. Special consideration is given to both normal operating conditions and peak-load situations, such as motor startup and stall conditions, to ensure that all components function within their specified limits and that the overall system operates efficiently and safely.

## 4.1 Power Supply Calculations

### Battery Configuration

The system is powered using four lithium-ion batteries, each with a nominal voltage of 3.7 V. The batteries are connected in series to increase the total output voltage. The resulting nominal supply voltage is:

$$V\_total = 3.7\ V \times 4 = 14.8\ V$$

### Voltage Regulation

The DC motors are rated for 12 V, but the L298 motor driver typically introduces a voltage drop of around 2–4 V; the motors therefore operate close to their rated capability rather than at the full battery voltage. The ESP32 operates at 3.3 V internally but accepts a 5 V input and regulates it internally.

### Current and Power Requirements

The current consumption of the electrical components is estimated based on datasheet values and typical operating conditions.

**Motor current (normal operation).** Each DC motor has a rated current of 0.3 A at 12 V. Since two motors are used:

$$I\_motors = 0.3\ A \times 2 = 0.6\ A$$

The rated electrical power consumed by the motors is:

$$P\_motors = V \times I = 12\ V \times 0.6\ A = 7.2\ W$$

**Motor current (stall condition).** The stall current of each motor is 2 A. Under worst-case conditions:

$$I\_stall = 2\ A \times 2 = 4\ A$$

The maximum power drawn during stall conditions is:

$$P\_stall = 12\ V \times 4\ A = 48\ W$$

Stall conditions are considered transient and are avoided during normal operation through appropriate control logic.

**ESP32 and logic circuit consumption.** The ESP32 microcontroller draws a current of approximately 240 mA during normal operation. The L298 motor driver also requires a small current for its internal logic circuits, approximately 40 mA. The total current required by the control and logic circuits is therefore:

$$I_{logic} = 0.24\ A + 0.04\ A = 0.28\ A$$

This current is supplied through a regulated low-voltage power source to ensure stable and reliable operation of the control electronics.

### Total Current Requirement

Under normal operating conditions:

$$I_{total} = I_{motors} + I_{logic} = 0.6\ A + 0.28\ A = 0.88\ A$$

Applying a safety factor of 30%:

$$I_{required} = 0.88\ A \times 1.3 \approx 1.15\ A$$

The battery pack and power-distribution system are therefore designed to safely supply currents exceeding this value.

## 4.2 Electrical and Electronics Calculations

The DC motor is the main actuator in this system. To control the motor's speed and direction, an L298 dual H-bridge motor driver is used as the interface between the ESP32 microcontroller and the DC motors.

Section 4.1 established the system's current and power budget from first principles. This section verifies that the selected L298 driver can meet that budget at the component level.

From the motor datasheet, the rated current of each motor under normal load is 0.3 A at 12 V, giving a combined rated current of 0.6 A and a rated power of 7.2 W (as calculated in Section 4.1). Under stall conditions, each motor draws up to 2 A, giving a combined stall current of 4 A and a peak power draw of 48 W.

The L298 motor driver is rated to supply a maximum continuous current of 2 A per channel. Since each motor is driven on its own channel, the driver comfortably meets the 0.3 A normal-operation requirement per motor, and can also tolerate the 2 A per-motor stall current for short durations without exceeding its rated limit. Continuous operation at stall current is avoided in the control logic to prevent overheating and long-term damage to the driver.

Combined with the ESP32's 240 mA draw, the total system current requirement of approximately 0.88 A (1.15 A with a 30% safety margin, as derived in Section 4.1) falls well within the supply capability of the four-cell 14.8 V battery pack regulated through the L298 driver.

# Chapter 5: Control Design

The control design of the differential-drive sumo robot defines how sensory information is processed and translated into motor commands to achieve stable, responsive, and competitive motion. The robot follows a differential-drive configuration, where two independently driven wheels allow linear and rotational movement through velocity control. The control system integrates mathematical modeling, algorithm development, and mechatronic simulation to validate system performance before physical implementation. Detailed calculations, parameter tuning, and implementation code are provided in the appendices.

## 5.1 Simulation of Mathematical Modeling (Simulink)

The mathematical model of the differential-drive sumo robot was developed to describe the kinematic behaviour of the system and to predict its motion under various control inputs. The model is based on the differential-drive kinematics derived in Section 3.2, where the robot's linear and angular velocities are functions of the left and right wheel angular velocities.

Using MATLAB/Simulink, a dynamic simulation environment was created to represent the robot's motion in a two-dimensional plane. The model incorporates wheel radius, wheelbase distance, motor angular velocity, and robot orientation. By simulating different velocity inputs, the robot's forward motion, turning behaviour, and rotational response were analysed.

The Simulink model enabled verification of the control strategy by observing position, orientation, and velocity responses over time. This simulation step ensured that the control logic produced stable motion and correct turning behaviour before implementation in hardware.

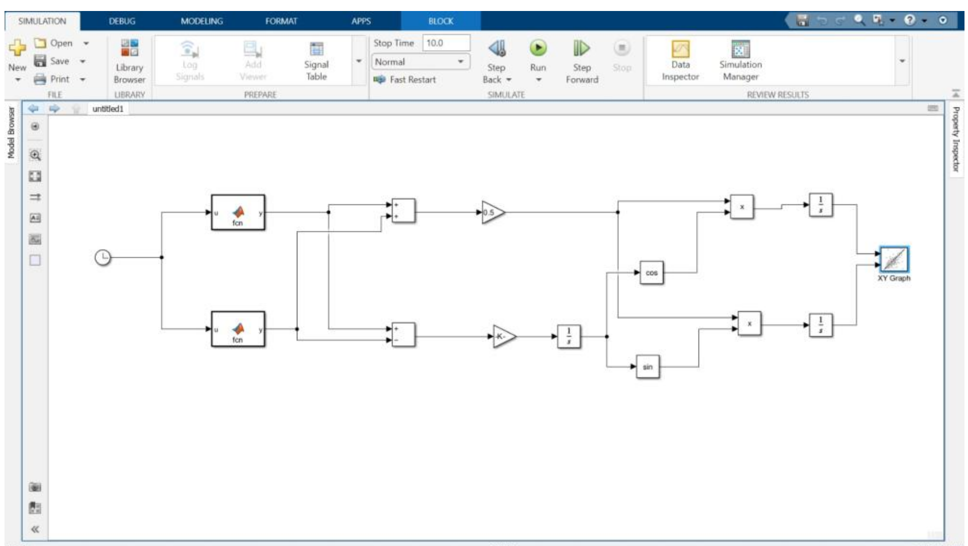



**Figure 5.1 — Simulink block diagram of the differential-drive kinematic model**

## 5.2 Control Algorithm (Flowchart)

The control algorithm governs the robot's decision-making process and movement strategy during operation. It is designed to process Bluetooth command input, determine the robot's current state, and generate the appropriate motor commands for navigation and opponent interaction.

The algorithm begins with system initialization, followed by continuous polling for an incoming Bluetooth command. Once a command is received, conditional logic determines whether the robot should move forward, move backward, or turn, and motor speed and direction are updated accordingly to achieve the desired motion.

A flowchart representation was used to clearly define the logical sequence and decision branches of the control algorithm, improving clarity, debugging, and future modification. The complete flowchart is shown below, and the corresponding implementation is provided in Appendix D.

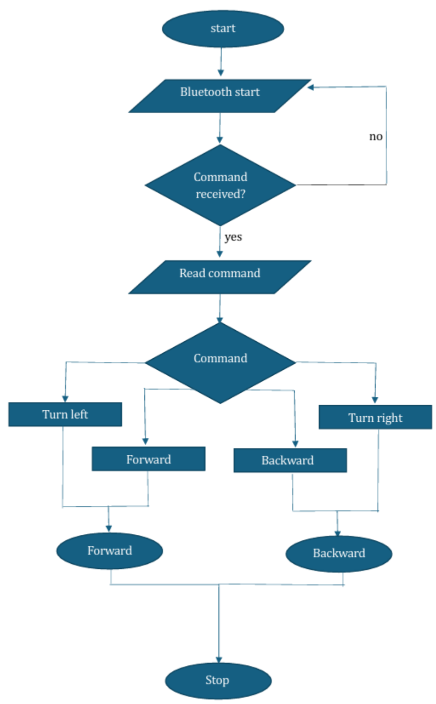



**Figure 5.2 — Control algorithm flowchart for Bluetooth command handling**

## 5.3 Mechatronics Simulation (Proteus)

Mechatronic simulation was carried out using Proteus to validate the integration of electronic components and control logic. The simulation environment included the microcontroller, motor driver, DC motors, and power supply, allowing the system to be tested virtually as a complete unit.

Proteus simulation enabled verification of signal flow between the microcontroller and actuators, ensuring correct motor driver operation and proper response to control signals. It also allowed observation of voltage levels, motor behaviour, and system timing under different control conditions.

By simulating the circuit and embedded control program prior to physical construction, potential hardware and software issues were identified and resolved early in the design process. The circuit schematic is provided in Appendix C.

# Chapter 6: Results and Discussion

The Sumo Robot has a total mass of approximately **1.5 kg** and dimensions of **15 cm × 20 cm**. The robot is manually controlled using an **ESP32 microcontroller** connected to a Bluetooth car-controller app, **two DC worm-gear motors**, a **motor driver**, **two wheels of 65 mm diameter**, and a **3D-printed body**.

## Maximum Robot Speed

The maximum linear speed of the robot depends on the motor speed and wheel diameter. The robot speed is calculated using:

$$V = (RPM \times \pi \times D) / 60$$

$$V = (32 \times 3.14 \times 0.065) / 60$$

$$V = 0.109\ m/s$$

The maximum robot speed is approximately **0.11 m/s**, which provides controlled and stable motion suitable for manual Sumo Robot operation.

## Response Delay Analysis

The total response delay of the robot is the sum of the wireless communication delay, microcontroller processing time, motor driver response, and motor mechanical response:

$$\text{total delay} = \text{wireless delay} + \text{processing delay} + \text{driver delay} + \text{motor delay}$$

$$\text{total delay} = 0.10 + 0.02 + 0.03 + 0.10 = 0.25\ s$$

The response delay is approximately **0.25 seconds**, which allows fast and reliable manual control.

## Pushing Capability

Although the motors provide high torque (**40 kg·cm**), the maximum pushing force of the robot is limited by wheel-to-ground traction, which mainly depends on robot weight. The maximum pushing force is given by:

$$F\_max = g \times mass \times \mu$$

$$F\_max = 9.81 \times 1.5 \times 0.5$$

$$F\_max \approx 7.36\ N$$

This force corresponds to pushing an opposing mass of approximately 1.5 kg.

### Real Pushing Test

In a real pushing test, the robot was unable to push a 7 kg object, and its wheels began to slip. The force required to push a 7 kg object is:

$$F\_required = 9.81 \times 7 \times 0.5 = 43.3\ N$$

Since the robot can only generate 7.36 N — roughly a fifth of the 43.3 N required — the wheels lost traction and slipped well before the object could be moved.

## Discussion

The results show that robot performance is mainly limited by traction rather than motor torque. The calculated speed and response delay are both well suited to manual control, while pushing capability could be improved by increasing the robot's weight or wheel friction. Although the motors provide high torque, the actual pushing force of the robot is constrained by its weight and the wheel-to-ground traction available, not by the motors themselves.

To increase pushing capability, the following solutions can be applied:

- Increase robot weight.
- Improve wheel friction (better tyres).
- Optimize weight distribution over the wheels.

These improvements would allow the robot to push heavier opponents more effectively during Sumo matches. Overall, the experimental results are consistent with the theoretical calculations.

# Chapter 7: Conclusions and Future Work

## 7.1 Conclusions

The design is successful due to its excellent stability and strong structural integrity. The robust 3D-printed chassis provides high resistance to external forces, making the robot difficult to push during competition. Additionally, the compact structure and low center of gravity improve balance and prevent tipping during collisions. Overall, the design performs reliably under the given constraints and meets the project requirements.

## 7.2 Recommendations

Future enhancements could focus on:

- **Improved opponent detection:** integrating distance sensors or vision-based systems, such as cameras or LiDAR, to enhance awareness of the opponent's position.
- **Enhanced mobility and power:** using higher-torque motors to increase pushing capability and overall movement efficiency.
- **Better traction and stability:** employing wheels with stronger grip and higher friction to improve ground adherence and prevent slipping during matches.

# Appendix A: Project Outline Details

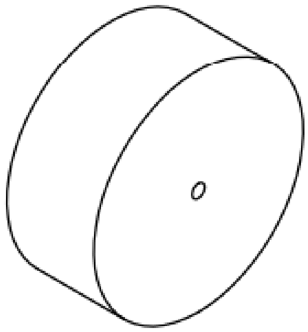
## Electronics Subsystem

- ESP32 (microcontroller)
- L298 motor driver
- Breadboard, 400 tie-point
- Battery charger, 1-cell, 3.7 V
- Switch
- Jumper wires

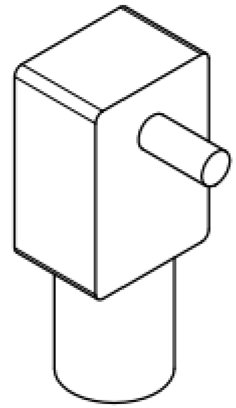
## Mechanical Subsystem

- JGY-370-1258 miniature gear motor, 12 V
- Robot wheels (rubber tyres)
- 3D-printed chassis

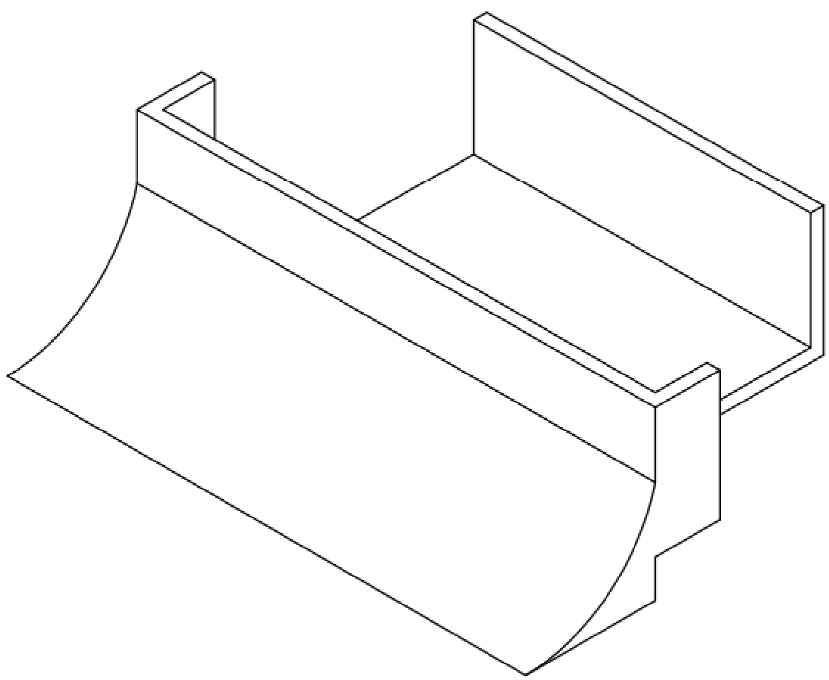
## Appendix B: Mechanical Drawings



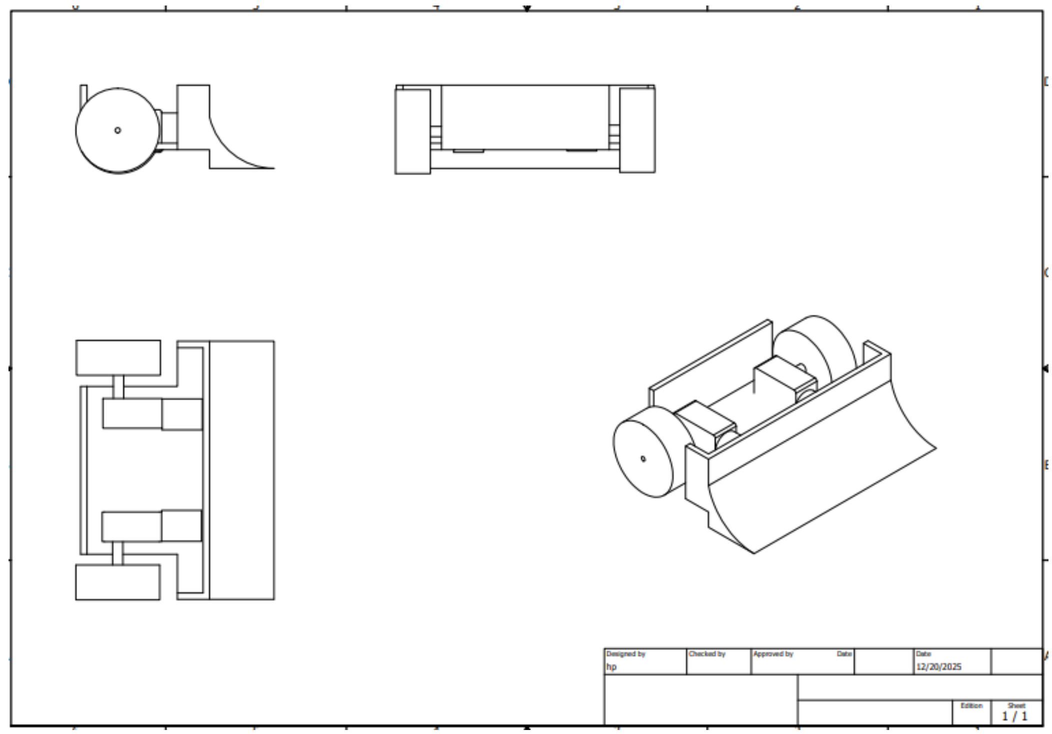
**Robot Wheel**



**JGY-370-1258 Miniature Gear Motor, 12 V**



**3D-Printed Chassis**




**Full Assembly — Orthographic and Isometric Views**

# Appendix C: Electrical Drawings

The full circuit schematic below shows the wiring between the ESP32 DevKit, the L298 motor driver, the two DC motors, the buzzer, and the 14.8 V battery pack.

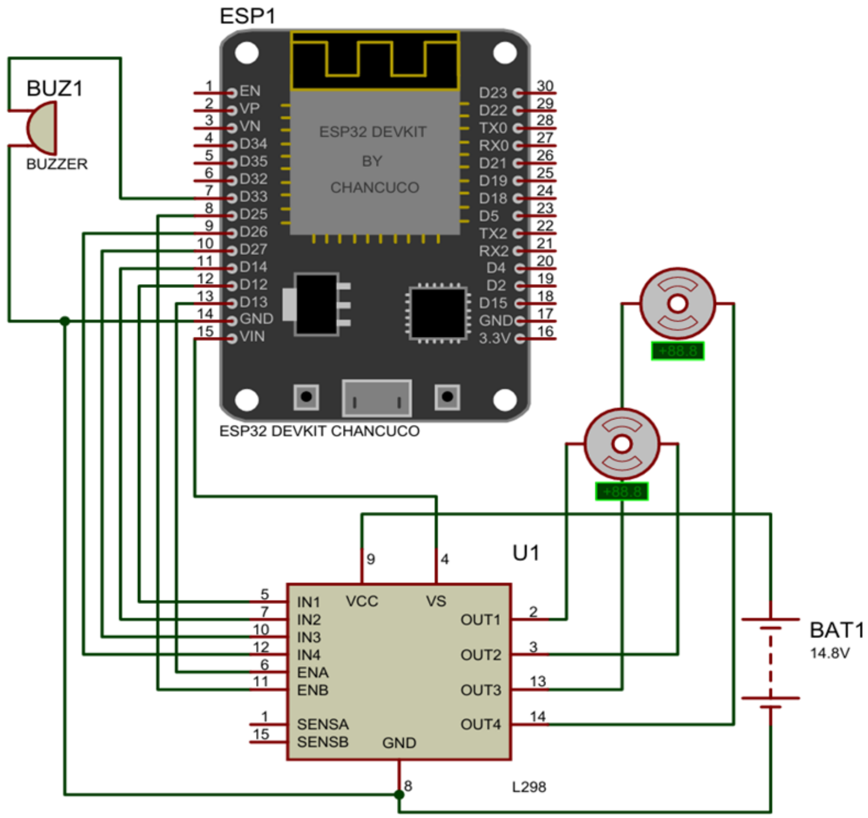



Figure C.1 — Full circuit schematic: ESP32, L298 motor driver, motors, and battery pack

# Appendix D: Programming Code Listing

The code consists of three logical parts running on the ESP32 board: Bluetooth command reception, motor driver control, and buzzer feedback. This code controls the movement of the sumo robot using Bluetooth communication. The ESP32 receives commands from a Bluetooth joystick app to determine the robot's direction and speed. Based on the received data, the motor driver controls the left and right motors to move forward, backward, or turn. Additionally, a buzzer is activated to provide audio feedback when a specific command is received, confirming successful communication.

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

// ---------- PIN MAPPING ----------
const int ENA = 13;
const int IN1 = 12;
const int IN2 = 14;
const int IN3 = 27;
const int IN4 = 26;
const int ENB = 25;
const int buzzer = 33;   // Buzzer Pin

// ---------- SETUP ----------
void setup() {
  Serial.begin(115200);
  SerialBT.begin("Jason_Joystick");

  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(buzzer, OUTPUT);

  // Startup Beep
  tone(buzzer, 2500, 200);
}

// ---------- MAIN LOOP ----------
void loop() {
  if (SerialBT.available()) {
    String data = SerialBT.readStringUntil('\n');
    data.trim();
    data.toUpperCase();

    // ----- BUZZER COMMAND -----
    if (data.indexOf('Y') > -1) {
```

```
      tone(buzzer, 2500, 500);
    }

    if (data.length() >= 6) {
      char driveDir = data[0];
      int driveSpeed = data.substring(1, 3).toInt();
      char turnDir = data[3];
      int turnAmount = data.substring(4, 6).toInt();

      int maxSpeed = 255;
      int drivePWM = map(driveSpeed, 0, 99, 0, maxSpeed);
      int turnPWM  = map(turnAmount, 0, 99, 0, maxSpeed);

      int leftSpeed, rightSpeed;

      if (driveSpeed < 10 && turnAmount > 10) {
        if (turnDir == 'R') {
          setDirections('B', 'F');
          leftSpeed = turnPWM;
          rightSpeed = turnPWM;
        } else {
          setDirections('F', 'B');
          leftSpeed = turnPWM;
          rightSpeed = turnPWM;
        }
      }
      else {
        char actualDir = (driveDir == 'F') ? 'B' : 'F';
        setDirections(actualDir, actualDir);

        if (turnDir == 'L') {
          leftSpeed  = drivePWM;
          rightSpeed = drivePWM - turnPWM;
        } else {
          leftSpeed  = drivePWM - turnPWM;
          rightSpeed = drivePWM;
        }
      }

      analogWrite(ENA, constrain(leftSpeed, 0, maxSpeed));
      analogWrite(ENB, constrain(rightSpeed, 0, maxSpeed));

      if (driveSpeed == 0 && turnAmount == 0)
        stopMotors();
    }
  }
}

// ---------- MOTOR DIRECTION ----------
```

```
void setDirections(char motLeft, char motRight) {
  if (motLeft == 'F') {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
  } else {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
  }

  if (motRight == 'F') {
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
  } else {
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
  }
}

// ---------- STOP MOTORS ----------
void stopMotors() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}
```

## Appendix E: Main Datasheets

The following sources and datasheets were used during the design and calculation process:


- *Journal:* International Journal of Innovations in Engineering Research and Technology (IJIERT), Volume 2, Issue 6, June 2015, Novateur Publications, Maharashtra, India.
- L298N Motor Driver datasheet — handsontec.com
- ESP32 datasheet — Espressif Systems documentation